What does scalability mean for web development?

Lecture 1 · Establish

Pressure changes. What must the system continue to do?

Scalability is a relationship, not a size

Pressure changes

Users, data, contributors, or requirements grow or vary.

Expected behavior remains bounded

The system preserves an explicit quality, cost, reliability, or change goal.

> “Scales well” is incomplete until both sides of the relationship are named.

Four pressures organize the course

Pressure	What changes?	Example question
Demand	Requests and useful work	Can the application sustain a tenfold traffic burst?
Data	Volume, velocity, shape, or retention	Can queries remain useful as history grows?
Contributor	People and agents changing the system	Can more contributors work without integration collapse?
Change	Requirements, dependencies, and environments	Can the system evolve without making every release risky?

Related qualities answer different questions

Performance asks how the system behaves under stated conditions.

Elasticity asks how capacity adapts to changing demand.

Resilience asks how behavior changes when parts fail.

Efficiency asks what resources an accepted outcome consumes.

Scalability connects changing pressure to the behavior that must be maintained.

Mechanisms are responses—not dimensions

Architecture and infrastructure give us response surfaces:

- move or duplicate computation;
- cache, replicate, partition, archive, or delete data;
- isolate failure and ownership boundaries;
- automate validation and deployment;
- change contracts, workflows, or responsibility.

Every response introduces a cost, constraint, or new failure mode.

Turn an ambition into a bounded claim

> Under **[declared pressure and conditions]**, the system maintains **[observable behavior]** within **[threshold or comparison]**, except for **[known limit]**.

Weak: “The application is scalable.”

Stronger: “With 20 concurrent users creating events, p95 accepted-write latency remains below 500 ms on the declared environment, until the database write path saturates.”

One product, three development systems

The course builds the same Work Activity Tracker three times:

1. **Establish:** local model, single-agent workflow.
2. **Challenge:** foundation model, single-agent workflow.
3. **Compare:** declared agent fleet.

The shared behavior stays stable so differences in architecture, evidence, and workflow become discussable.

Establish checkpoint

Before the next lecture, produce:

- two bounded scalability claims;
- the project's important constraints;
- an initial implementation A hypothesis;
- one pressure map for the application and development system.

Question to carry forward: What evidence would make your claim credible?